

PROJECT REPORT
UMU Online Voting System for Student Guild Elections

REGISTRATION NUMBERS

1. 2023-B072-31712
2. 2023-B201-31574

1. System Description

1.1 Overview

The UMU Online Voting System is a secure, browser-based application that allows Uganda Martyrs University students to participate in Student Guild elections entirely online. The system eliminates manual ballot papers, reduces administrative overhead and provides real-time election results. It was built using PHP 8 with PDO for database access, a MySQL backend, and Vanilla JavaScript with CSS3 for the frontend all technologies covered in.

1.2 Key Features

- Secure student self-registration with UMU email validation (@stud.umu.ac.ug) and registration-number format enforcement.
- Admin approval workflow: newly registered students remain inactive until an administrator approves their accounts.
- Authenticated ballot page where each approved student may cast exactly one vote per position, enforced at both application and database level.
- Live results dashboard accessible to administrators showing vote tallies per candidate, grouped by position.
- Full admin control panel: manage positions, candidates, user accounts, and a voting open/closed toggle.
- Role-based access control separating student and admin capabilities throughout the application.

1.3 User Roles

The system defines two roles:

- Student registers, logs in, casts one vote per position, and views their submitted ballot.
- Admin approves students, manages candidates and positions, views real-time results, and controls election settings.

2. Database Structure

The application uses a relational MySQL database named umu_voting. The schema consists of five tables with well-defined foreign-key relationships that enforce referential integrity and prevent data anomalies such as orphaned votes.

Table	Column	Type	Description
users	id	INT (PK)	Auto-increment primary key
	reg_number	VARCHAR(20)	Unique student number e.g. 2023-B072-31711
	full_name	VARCHAR(120)	Student full name
	email	VARCHAR(160)	University email (unique)
	password	VARCHAR(255)	Bcrypt-hashed password
	role	ENUM	'student' or 'admin'
	is_approved	TINYINT(1)	Admin approval flag (0 = pending)
	has_voted	TINYINT(1)	Prevents double-voting
positions	photo_path	VARCHAR(255)	Relative path to uploaded photo
	id	INT (PK)	Primary key
	title	VARCHAR(100)	e.g. Guild President

	display_order	TINYINT	Sort order on ballot
candidates	id	INT (PK)	Primary key
	position_id	INT (FK)	Links to positions.id
	full_name	VARCHAR(120)	Candidate name
	manifesto	TEXT	Candidate's campaign statement
	vote_count	INT	Running tally of votes received
votes	id	INT (PK)	Primary key
	user_id	INT (FK)	Links to users.id
	candidate_id	INT (FK)	Links to candidates.id
	position_id	INT (FK)	Ensures one vote per position
	voted_at	TIMESTAMP	Audit timestamp
settings	setting_key	VARCHAR(60)	e.g. 'voting_open', 'election_title'
	setting_value	VARCHAR(255)	Value for the setting

2.1 Key Relationships

- votes.user_id: users.id (one student, many votes across positions)
- votes.candidate_id: candidates.id (one candidate receives many votes)
- votes.position_id: positions.id with UNIQUE(user_id, position_id) database-level prevention of double-voting per position.
- candidates.position_id: positions.id ON DELETE CASCADE removing a position removes its candidates and all associated votes automatically.

3. Functional Requirements Coverage

The table below maps each of the ten required functional capabilities defined in the project brief to the specific implementation in the UMU Voting system.

	Requirement	Implementation in UMU Voting
1	User Registration & Login	register.php validates UMU email format and reg number pattern, stores bcrypt password, creates pending account. index.php handles PDO login.
2	Logout & Session Control	logout.php destroys the session. All protected pages call session_start() and redirect unauthenticated users to index.php.
3	Data Entry through Forms	Students submit votes via vote.php; admins add candidates, positions and manage settings through forms in admin/.
4	Input Validation with Feedback	Server-side validation on all forms with inline error messages. Invalid submissions are rejected and data is not stored.
5	Dynamic Data Display	Candidates and positions are fetched from the database via PDO queries and rendered dynamically on the ballot and results pages.

6	Search or Filter	admin/users.php provides real-time search/filter of student accounts. admin/results.php displays filtered vote tallies by position.
7	Editing Records	Admin can edit candidate details, position titles and system settings (election title, voting open/closed) through admin panel forms.
8	Deleting Records	Admin can delete candidates and user accounts. Cascade foreign keys in the schema remove dependent vote records automatically.
9	Role-Based Access	Two roles: student and admin. Admin pages in admin/ verify role === 'admin' before rendering. Students cannot access admin URLs.
10	Persistent Data Storage	All data (users, votes, candidates, settings) is stored in a MySQL database via PDO. Data survives server restarts and page refreshes.

4. Challenges and Reflections

4.1 Challenges Faced

- Double-vote prevention: Implementing a single-vote-per-position constraint required both application-level checks (has_voted flag) and a UNIQUE composite key (user_id, position_id) in the votes table to handle concurrent submissions safely.
- Admin approval workflow: Coordinating the is_approved flag across registration, login and the admin users panel required careful session and redirect logic to avoid approved users being locked out or pending users slipping through.
- Photo upload and display: Handling file validation (type, size), generating unique filenames to avoid collisions, and serving uploaded images relative to the web root introduced several path-resolution edge cases across different WAMP configurations.
- Schema evolution during development: Adding the photo_path column to the candidates table after initial database creation caused a PDO column-not-found error on the live server where the older schema was already imported. This was resolved by issuing an ALTER TABLE statement to bring the database in sync with the latest schema.
- Role-based redirects: Ensuring that students who attempt to access admin URLs are redirected cleanly, and that admin users are not directed to the student ballot, required consistent guard checks at the top of every protected page.

Screenshots

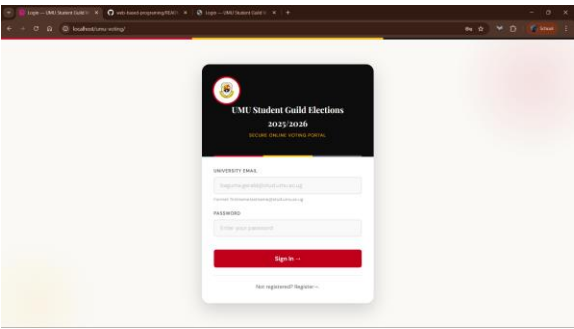


Figure 1. login page

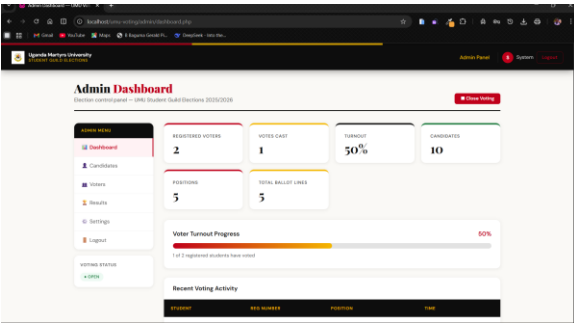


Figure 3. admin dashboard

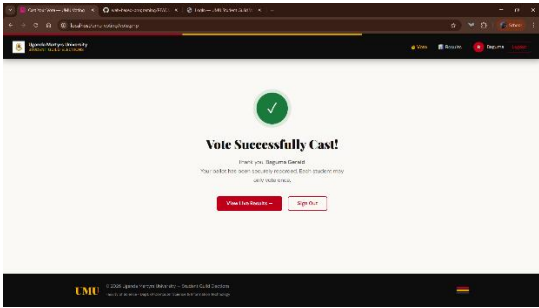


Figure 2. after casting the vote

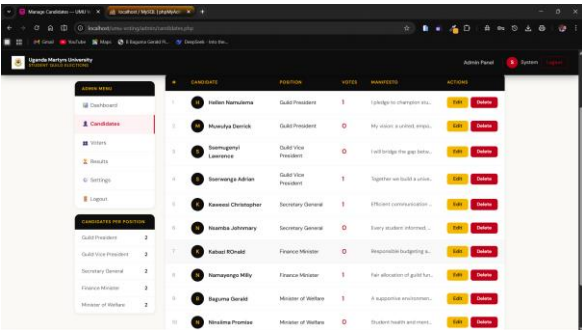


Figure 4. candidates page

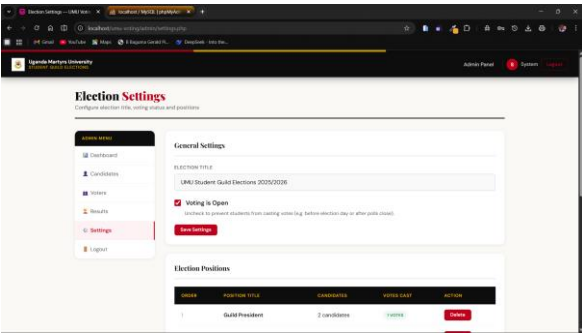


Figure 5. setting page